ExLab03

```
/* Sã se ob?inã o listã cu urmãtorul con?inut (pentru fiecare
angajat):
 - nume SEF = concatenarea prenumelui cu numele (este sef cine are
minim un subaltern)
 - venit minim pentru departamentul in care e afla respectivul sef
 - venit maxim pentru departamentul in care e afla respectivul sef
 - daca este peste salariul sefului de departement din departamentul
lui = 'DA/NU'
 - primul departament cu salariu mai mare ca a lui** (oportunitatea de
de promovare)

**: gasiti primul salariu mai mare strict ca al lui pentru un
departament care nu este a lui
*/

SET SERVEROUTPUT ON;
DECLARE
    TYPE sef IS RECORD(
        nume_sef VARCHAR2(46),
        venit_minim NUMBER,
        venit_maxim NUMBER,
        e_peste VARCHAR2(2),
        id_dept departments.department_id%TYPE
    );
    TYPE tablenested IS TABLE OF sef;

    lista tablenested;
    contor INTEGER;
BEGIN
    SELECT m.first_name || ' ' || m.last_name,
        msts.min_venit,
        msts.max_venit,
        CASE WHEN (m.salary >= (SELECT SALARY FROM employees WHERE
employee_id = (SELECT manager_id
                FROM departments
                WHERE department_id = m.department_id))) THEN 'DA'
            ELSE 'NU'
        END  e_peste,
        (SELECT department_id
        FROM employees
        WHERE salary = (SELECT MIN(salary)
            FROM employees
            WHERE salary > m.salary
                AND m.department_id <> department_id)
            AND department_id <> m.department_id
        ORDER BY department_id
        FETCH NEXT 1 ROWS ONLY) id_dep
    bulk collect into lista
    FROM employees m
```

```
    INNER JOIN (SELECT department_id,
            MIN(salary * (1 + NVL(commission_pct, 0))) min_venit,
            MAX(salary * (1 + NVL(commission_pct, 0))) max_venit
        FROM employees
        GROUP BY department_id) msts ON m.department_id =
msts.department_id
    WHERE m.employee_id IN (SELECT manager_id FROM employees);

    /*
    TYPE sef IS RECORD(
        nume_sef VARCHAR2(46),
        venit_minim NUMBER,
        venit_maxim NUMBER,
        e_peste VARCHAR2(2),
        id_dept departments.department_id%TYPE
    );
    */
    FOR contor IN lista.FIRST..lista.LAST
    LOOP
        dbms_output.put_line(RPAD(lista(contor).nume_sef, 46) ||
RPAD(lista(contor).venit_minim, 10));
    END LOOP;
END;
```

ExLab03-v2

```
/* Sa se obtina o lista cu urmatorul continut (pentru fiecare
angajat):
 - nume functie
 - diferenta intre valoare minima a functiei si valoarea minima reala
a functiei curente
 - diferenta intre valoare maxima a functiei si valoarea maxima reala
a functiei curente
 - mediana salariului pentru functia respectiva
*/

DECLARE
    TYPE info_functie IS RECORD(
        id_job jobs.job_id%TYPE,
        nume_functie jobs.job_title%TYPE,
        diff_min employees.salary%TYPE,
        diff_max employees.salary%TYPE
    );
    TYPE lista IS TABLE OF info_functie;
    TYPE lista_ordonata IS TABLE OF employees.salary%TYPE;
```

```
    l_functii lista;
    l_ord lista_ordonata;
BEGIN
    SELECT j.job_id,
        j.job_title,
        MIN(e.salary) - j.min_salary diff_min,
        j.max_salary - MAX(e.salary) diff_max
    BULK COLLECT INTO l_functii
    FROM employees e
    JOIN jobs j on e.job_id = j.job_id
    GROUP BY j.job_id, j.job_title, j.max_salary, j.min_salary;

    FOR contor in l_functii.FIRST..l_functii.LAST
    LOOP
        SELECT salary
        BULK COLLECT INTO l_ord
        FROM employees
        WHERE job_id = l_functii(contor).id_job
        ORDER BY salary;

        dbms_output.put_line(RPAD(l_functii(contor).nume_functie, 40)
        || RPAD(l_functii(contor).diff_min, 10)
        || RPAD(l_functii(contor).diff_max, 10)
        || RPAD(l_ord(l_ord.LAST / 2), 10));
    END LOOP;
EXCEPTION
    WHEN no_data_found THEN
        dbms_output.put_line('Nu au fost gasiti angajati');
END;
```

ExLab03-v3

```
/* S? se obtin? o list? cu urm?torul continut (pentru fiecare
angajat):
 - nume tara
 - numarul de functii distincte din acea tara
 - functia cu venit maxim pentru tara
 - functia cu venit minim pentru tara
 - cati sefi de departament are respectiva tara
 - prima tara cu venit mediu mai mare
*/
DECLARE
    TYPE info_tara IS RECORD(
        nume_tara countries.country_name%TYPE,
        nr_functii INTEGER,
        venit_maxim NUMBER(8,2),
        venit_minim NUMBER(8,2),
        nr_sefi INTEGER,
```

```
        venit_mediu NUMBER(8,2)
    );
    TYPE tabel_tari IS TABLE OF info_tara;

    lista tabel_tari;
    functie_max jobs.job_title%TYPE;
    functie_min jobs.job_title%TYPE;
    tara_v countries.country_name%TYPE;
BEGIN
    SELECT c.country_name,
        COUNT(DISTINCT e.job_id),
        NVL(MAX(e.salary *(1 + nvl(e.commission_pct, 0))), 0),
        NVL(MIN(e.salary *(1 + nvl(e.commission_pct, 0))), 0),
        COUNT(DISTINCT d.manager_id),
        NVL(AVG(e.salary *(1 + nvl(e.commission_pct, 0))), 0)
    BULK COLLECT INTO lista
    FROM countries c
    LEFT JOIN locations l ON c.country_id = l.country_id
    LEFT JOIN departments d ON l.location_id = d.location_id
    LEFT JOIN employees e ON d.department_id = e.department_id
    GROUP BY c.country_name
    ORDER BY 6 ASC;

    FOR contor IN lista.FIRST..lista.LAST
    LOOP
        BEGIN
            SELECT j.job_title
            INTO functie_max
            FROM jobs j
            NATURAL JOIN employees e
            WHERE lista(contor).venit_maxim = e.salary *(1 +
nvl(e.commission_pct, 0));

            SELECT j.job_title
            INTO functie_min
            FROM jobs j
            NATURAL JOIN employees e
            WHERE lista(contor).venit_minim = e.salary *(1 +
nvl(e.commission_pct, 0));

            FOR contor2 IN (contor + 1)..lista.LAST
            LOOP
                BEGIN
                    IF (lista(contor).venit_mediu <
lista(contor2).venit_mediu) THEN
                        tara_v := lista(contor2).nume_tara;
                        EXIT;
                    END IF;
                EXCEPTION
                    WHEN OTHERS THEN
                        tara_v := '';
                END;
```

```
            END LOOP;

            dbms_output.put_line(RPAD(lista(contor).nume_tara, 50) ||
RPAD(lista(contor).nr_functii, 15)|| RPAD(lista(contor).nr_sefi, 15)
            || RPAD(functie_max, 46)|| RPAD(functie_min, 46)||
RPAD(tara_v, 46));
        EXCEPTION
            WHEN OTHERS THEN
                 dbms_output.put_line('Nu ne place de ' ||
RPAD(lista(contor).nume_tara, 50));
        END;
    END LOOP;
END;
```

ExLab03-v5

```
/* Nota: Trebuie rulata urmatoarea instructiune o data inainte de
exercitiu
INSERT INTO job_history
SELECT employee_id,
    HIRE_DATE+1,
    SYSDATE,
    job_id,
    department_id
FROM employees;

Sa se obtina o lista cu urmatorul continut (pentru fiecare angajat):
 - nume functie (nume job)
 - cati angajati si-au schimbat jobul (pentru randul curent)
 - cati aganajati si-au schimbat jobul in alt departament decat cel
initial(pentru randul curent)
 - cati angajati au avut minim o crestere salaraiala pe parcusul
carierei (pentru randul curent)
 - numarul maxim de schimbari de job pentru cineva cu acest job
 - numarul minim de schimbari de job pentru cineva cu acest job
*/

SELECT a.job_title,
    SUM(a.nr_schimbari) nr_schimbari,
    SUM(b.nr_schimbari) nr_schimbari_dept,
    SUM(CASE WHEN c.max_sal > c.min_sal THEN 1 ELSE 0 END) nr_cresteri
FROM(SELECT j.job_id,
        j.job_title,
        jh.employee_id,
        CASE WHEN COUNT(*) >=2 THEN 1 ELSE 0 END nr_schimbari
```

```
    FROM jobs j
    JOIN job_history jh ON j.job_id = jh.job_id
    GROUP BY j.job_id, j.job_title, jh.employee_id) a
INNER JOIN (SELECT jh.employee_id,
            jh.job_id,
            CASE WHEN COUNT(DISTINCT jh.department_id) >=2 THEN 1 ELSE
0 END nr_schimbari
        FROM job_history jh
        GROUP BY jh.employee_id, jh.job_id) b ON a.job_id = b.job_id
AND a.employee_id = b.employee_id
INNER JOIN (SELECT j.job_id,
        jh.employee_id,
        MIN(j.max_salary) min_sal,
        MAX(j.max_salary) max_sal
    FROM jobs j
    JOIN job_history jh ON j.job_id = jh.job_id
    GROUP BY j.job_id, jh.employee_id) c ON c.job_id = b.job_id AND
c.employee_id = b.employee_id
GROUP BY a.job_title;

DECLARE
    TYPE info_functie IS RECORD(
        nume_f jobs.job_title%TYPE,
        nr_schimbari INTEGER,
        nr_schimbari_dept INTEGER,
        nr_cresteri INTEGER,
        nr_max INTEGER,
        nr_min INTEGER
    );
    TYPE l_functie IS TABLE OF info_functie;

    l_f l_functie;
BEGIN
    dbms_output.put_line('');
END;
```

ExLab03-v6

```
/* Sa se obtina o lista cu urmatorul continut (pentru fiecare
angajat):
 - nume manager (are macar o persoana ca subaltern)
 - numar subalterni directi
 - numar subalterni total (adica ierarhic)
 - cel mai mare salariu al unui subaltern direct cu o functie diferita
da a lui
 - numar functii distincte peste care este manager (ierarhic)
```

```
*/

SET SERVEROUTPUT ON;

DECLARE
    TYPE tip_ang IS RECORD (
        id_ang       employees.employee_id%TYPE,
        nume_mgr     VARCHAR2(255),
        nr_directi   INTEGER,
        nr_ierarhici INTEGER,
        sal_max      employees.salary%TYPE,
        nr_functii   INTEGER
    );
    TYPE id_ang IS
        TABLE OF tip_ang INDEX BY BINARY_INTEGER;
    TYPE v_set IS
        TABLE OF BINARY_INTEGER INDEX BY BINARY_INTEGER;
    TYPE func_set IS
        TABLE OF BINARY_INTEGER INDEX BY VARCHAR2(10);
    visited      v_set;
    de_adaugat   v_set;
    l_bfs        v_set;
    functii      func_set;
    lista_id_ang id_ang;
    func_ang     VARCHAR2(10);
    id_curent    BINARY_INTEGER;
    nr_elem      BINARY_INTEGER;
BEGIN
    SELECT
        m.employee_id,
        m.first_name
        || ' '
        || m.last_name nume_mgr,
        COUNT(*)       nr_subalterni_d,
        0,
        MAX(
            CASE
                WHEN e.job_id <> m.job_id THEN
                    e.salary
                ELSE
                    0
            END
        )              sal_max,
        0
    BULK COLLECT
    INTO lista_id_ang
    FROM
             employees m
        JOIN employees e ON e.manager_id = m.employee_id
    GROUP BY
        m.employee_id,
        m.first_name
```

```
        || ' '
        || m.last_name;

    dbms_output.put_line(rpad('nume_mgr', 46)
                         || rpad('nr_directi', 15)
                         || rpad('nr_ierarhici', 15)
                         || rpad('sal_max', 15)
                         || lpad('nr_functii', 15));

    FOR counter IN lista_id_ang.first..lista_id_ang.last LOOP
        /*
            luam primul nivel pentru managerul curent, angajatii
directi
            si ii punem in lista de explorat
        */
        SELECT
            employee_id
        BULK COLLECT
        INTO l_bfs
        FROM
            employees
        WHERE
            manager_id = lista_id_ang(counter).id_ang;

        -- initialiam dependintele
        visited.DELETE();
        functii.DELETE();
        id_curent := 1;
        -- vizitam si adaugam functiile elementelor initiale
        FOR idx in l_bfs.FIRST .. l_bfs.LAST
        LOOP
            visited(l_bfs(idx)) := 1;

            SELECT job_id
            INTO func_ang
            FROM employees
            WHERE employee_id = l_bfs(idx);

            functii(func_ang) := 1;
        END LOOP;

        WHILE ( true ) LOOP
            BEGIN
                /*
                    nu punem conditia in while deoarece e posibil sa
actualizam
                    lista curenta
                */
                IF ( id_curent > l_bfs.count ) THEN
                    EXIT;
                END IF;
                -- avem nevoie sa stim de la ce indice adaugam
```

```
                nr_elem := l_bfs.count;

                BEGIN
                    /*
                        adaugam la o lista temporara subalternii
directi ai
                        angajatului curentm care a aparut in lista
datorita
                        unui angajat valid, dar nu stim daca este
adaugat
                        deja sau nu
                    */
                    SELECT
                        employee_id
                    BULK COLLECT
                    INTO de_adaugat
                    FROM
                        employees
                    WHERE
                        manager_id = l_bfs(id_curent); -- aici aveam
.id_ang, dar nu aveam o structura acolo...

                    -- ne asiguram ca adaugam doar angajati noi in
lista
                    FOR idx IN de_adaugat.first..de_adaugat.last LOOP
                        IF NOT visited.EXISTS(de_adaugat(idx)) THEN
                            nr_elem := nr_elem + 1;
                            l_bfs(nr_elem) := de_adaugat(idx);
                            visited(de_adaugat(idx)) := 1;

                            -- vedem ce functie are angajatul nou
adaugat
                            SELECT
                                job_id
                            INTO func_ang
                            FROM
                                employees
                            WHERE
                                employee_id = de_adaugat(idx);

                            -- punem sau actualizam functia
angajatului adaugat
                            functii(func_ang) := 1;
                        END IF;
                    END LOOP;
                EXCEPTION
                    WHEN OTHERS THEN
                        --No more elements
                        NULL;
                END;

                id_curent := id_curent + 1;
```

```
            EXCEPTION
                WHEN OTHERS THEN
                    dbms_output.put_line('I did an oups');
                    EXIT;
            END;
        END LOOP;

        dbms_output.put_line(rpad(lista_id_ang(counter).nume_mgr, 46)
                             || rpad(lista_id_ang(counter).nr_directi,
15)
                             || rpad(l_bfs.count, 15)
                             || rpad(lista_id_ang(counter).sal_max,
15)
                             || lpad(functii.count, 15));

    END LOOP;

END;
```

ExLab04-v1

```
/*
Sa se afiseze folosind o functie si o procedura stocata (minim)
urmatoarea lista:
- nume angajat
- daca contine 'A' in nume (nu la inceput sau final) (DA/NU)
- daca este angajat intr-o luna de vara (DA/NU)
- care este diferenta intre venitul lui si veninut urmator al unui
coleg
sub acelasi manager (inf si sup)
*/
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION F_areA(nume IN VARCHAR2) RETURN VARCHAR2
AS
BEGIN
    CASE INSTR(UPPER(nume), 'A', 2)
        WHEN 0 THEN RETURN 'NU';
        WHEN LENGTH(nume) THEN RETURN 'NU';
        ELSE RETURN 'DA';
    END CASE;
END;
/

CREATE OR REPLACE FUNCTION F_lunaDeVara(data_ang IN date) RETURN
VARCHAR2
```

```
AS
BEGIN
    IF EXTRACT(MONTH FROM data_ang) IN (6, 7, 8) THEN
        RETURN 'DA';
    ELSE
        RETURN 'NU';
    END IF;
END;
/

CREATE OR REPLACE FUNCTION F_diffSal(ven IN NUMBER, manager_id IN
NUMBER) RETURN VARCHAR2
AS
    TYPE sal_tab IS TABLE OF NUMBER;

    vect sal_tab;
    idx INTEGER;
BEGIN
    SELECT DISTINCT e.salary * (1 + nvl(e.commission_pct, 0))
    BULK COLLECT INTO vect
    FROM employees e
    WHERE e.manager_id = manager_id
    ORDER BY 1;

    IF vect.COUNT = 1 THEN
        RETURN '- / -';
    END IF;

    FOR idx IN vect.FIRST..vect.LAST
    LOOP
        IF vect(idx) = ven THEN
            CASE idx
                WHEN vect.FIRST THEN RETURN '- / ' || (vect(idx + 1) -
ven);
                WHEN vect.LAST THEN RETURN (ven - vect(idx - 1)) || '
/ -';
                ELSE RETURN (ven - vect(idx - 1)) || ' / ' ||
(vect(idx + 1) - ven);
            END CASE;
        END IF;
    END LOOP;

    RETURN '- / -';
END;
/

BEGIN
    FOR angajat IN (SELECT  first_name || ' ' || last_name as nume,
        F_lunaDeVara(hire_date) luna_vara,
        F_diffSal(salary * (1 + nvl(commission_pct, 0)), manager_id)
diff_ven
        FROM employees)
```

```
    LOOP
        dbms_output.put_line(RPAD(angajat.nume, 50) ||
RPAD(F_areA(angajat.nume), 10)|| RPAD(angajat.luna_vara, 10)||
LPAD(angajat.diff_ven, 20) );
    END LOOP;
END;
```

ExLab04-v2

```
/*
Sa se afiseze folosind o functie si o procedura stocata (minim)
urmatoarea lista:
- nume angajat
- daca lungimea numelui este prima
- care este diferenta intre salariul lui si cel mai mare salariu
pentru functia pe care o detine
- care este diferenta intre salariul lui si cel mai mic salariu pentru
functia pe care o detine
- cate alte joburi distincte sunt sub managerul lui
*/
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION F_LungimePrima(nume VARCHAR2) RETURN
VARCHAR2
AS
    lungime INTEGER := LENGTH(nume) - 1;
BEGIN
    FOR idx IN 2..(lungime/2)
    LOOP
        IF MOD(lungime, idx) = 0 THEN
            RETURN 'NU';
        END IF;
    END LOOP;

    RETURN 'DA';
END;

/

BEGIN
    FOR angajat IN (
        SELECT e.first_name||' ' || e.last_name nume,
            e.salary,
            s.sal_max,
            s.sal_min,
```

```
            (SELECT COUNT(DISTINCT job_id) FROM EMPLOYEES m WHERE
m.manager_id = e.manager_id) - 1 AS alte_joburi
        FROM EMPLOYEES e
        INNER JOIN (
            SELECT job_id,
                MAX(salary) sal_max,
                MIN(salary) sal_min
            FROM EMPLOYEES
            GROUP BY job_id) s ON s.job_id = e.job_id)
    LOOP
        dbms_output.put_line(RPAD(angajat.nume, 50)
            || RPAD(F_LungimePrima(angajat.nume), 10)
            || RPAD(angajat.sal_max - angajat.salary, 10)
            || RPAD(angajat.salary - angajat.sal_min, 10)
            || RPAD(angajat.alte_joburi, 10) );
    END LOOP;
END;
```

ExLab04-v3

```
/*
Sa se afiseze folosind o functie si o procedura stocata (minim)
urmatoarea lista:
- nume angajat
- daca este sef sau sef de departament (sa se poata faca distinctie
intre categorii)
- pentru functia lui care este diferenta medie fata de salariul maxim
real
- pentru departamentul lui care este diferenta medie fata de salariul
minim real
- cati subalterni directi cu lungimea numelui impara are
- primul departament cu mai multi angajati decat al lui
*/
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION F_ESef(id_ang NUMBER) RETURN VARCHAR2
AS
BEGIN
    FOR i IN (SELECT DISTINCT manager_id FROM departments WHERE
manager_id IS NOT NULL)
    LOOP
        IF id_ang = i.manager_id THEN
            RETURN 'E MARE SEF';
        END IF;
    END LOOP;

    FOR i IN (SELECT DISTINCT manager_id FROM employees WHERE
manager_id IS NOT NULL)
```

```
    LOOP
        IF id_ang = i.manager_id THEN
            RETURN 'E SEF';
        END IF;
    END LOOP;

    RETURN 'NU-I SEF';
END;
/

CREATE OR REPLACE FUNCTION F_LungimeImpara(id_ang NUMBER) RETURN
INTEGER
AS
    nr_ang INTEGER;
BEGIN
    SELECT COUNT(*)
    INTO nr_ang
    FROM employees
    WHERE manager_id = id_ang
        AND MOD(LENGTH(first_name||' ' ||last_name), 2) = 1;

    RETURN nr_ang;
END;
/

CREATE OR REPLACE FUNCTION F_UrmatorulDept(id_dept NUMBER) RETURN
INTEGER
AS
    FLAG INTEGER := 0;
BEGIN
    FOR x IN (
        SELECT DEPARTMENT_ID, COUNT(*)
        FROM EMPLOYEES
        GROUP BY DEPARTMENT_ID
        ORDER BY 2)
    LOOP
        IF FLAG = 1 THEN
            RETURN x.DEPARTMENT_ID;
        END IF;

        IF x.DEPARTMENT_ID = id_dept THEN
            FLAG := 1;
        END IF;
    END LOOP;

    RETURN -1;
END;
/

CREATE OR REPLACE PROCEDURE Afisare(nume VARCHAR2, id_ang NUMBER,
diff1 NUMBER, diff2 NUMBER, id_dept NUMBER)
AS
```

```
BEGIN
    dbms_output.put_line(RPAD(nume, 50)
        || RPAD(F_ESef(id_ang), 15)
        || RPAD(diff1, 10)
        || RPAD(diff2, 10)
        || RPAD(F_LungimeImpara(id_ang), 10)
        || LPAD(F_UrmatorulDept(id_dept), 10)
        );
END;
/

BEGIN
    dbms_output.put_line(RPAD('Nume', 50)
        || RPAD('E Sef', 15)
        || RPAD('Diff F', 10)
        || RPAD('Diff D', 10)
        || RPAD('Lungime Imp', 10)
        || LPAD('Dept Next', 10)
        );
    dbms_output.put_line(RPAD('*', 50, '*')
        || RPAD('*', 15, '*')
        || RPAD('*', 10, '*')
        || RPAD('*', 10, '*')
        || RPAD('*', 10, '*')
        || LPAD('*', 10, '*')
        );
    FOR angajat IN (
        SELECT e.first_name||' ' ||e.last_name nume,
            e.EMPLOYEE_ID,
            e.department_id,
            ROUND(j.max_sal - j.avg_sal) diff_medie,
            ROUND(d.max_sal - d.avg_sal) diff_medie2
        FROM EMPLOYEES e
        JOIN (SELECT job_id, MAX(salary) max_sal, AVG(salary) avg_sal
            FROM EMPLOYEES GROUP BY job_id) j ON j.job_id = e.job_id
        JOIN (SELECT department_id, MAX(salary) max_sal, AVG(salary)
avg_sal
            FROM EMPLOYEES GROUP BY department_id) d ON
d.department_id = e.department_id
    )
    LOOP
        Afisare(angajat.nume,
            angajat.EMPLOYEE_ID,
            angajat.diff_medie,
            angajat.diff_medie2,
            angajat.department_id);
    END LOOP;
END;
```

ExLab05-v1

```
/*
Sa se afiseze folosind o functie si o procedura stocata (minim)
urmatoarea lista:
- nume angajat
- daca numarul de subalterni directi este par
- daca functiile distincte subalternilor directi sunt in numar impar
- care este diferenta maxima de salariu din departamentul lui
- din cate tari distincte are subalterni
- care este salariul maxim pe care il poate obtine el sau un subaltern
*/
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION F_Subalterni(id_ang NUMBER)
RETURN VARCHAR2
AS
    nr_ang NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO nr_ang
    FROM employees
    WHERE manager_id = id_ang;

    IF nr_ang MOD 2 = 0 THEN
        RETURN 'PAR';
    END IF;

    RETURN 'IMPAR';
END;
/

CREATE OR REPLACE FUNCTION F_Functii(id_ang NUMBER)
RETURN VARCHAR2
AS
    nr_ang NUMBER;
BEGIN
    SELECT COUNT(DISTINCT job_id)
    INTO nr_ang
    FROM employees
    WHERE manager_id = id_ang;

    IF nr_ang MOD 2 = 0 THEN
        RETURN 'PAR';
    END IF;

    RETURN 'IMPAR';
END;
/
```

```
CREATE OR REPLACE FUNCTION F_DiffMax(id_dept NUMBER)
RETURN NUMBER
AS
    diff_max NUMBER;
BEGIN
    SELECT NVL(MAX(salary) - MIN(salary), 0)
    INTO diff_max
    FROM employees
    WHERE department_id = id_dept;

    RETURN diff_max;
END;
/

CREATE OR REPLACE FUNCTION F_TariDistincte(id_ang NUMBER)
RETURN NUMBER
AS
    nr_tari NUMBER;
BEGIN
    SELECT COUNT(DISTINCT locations.country_id)
    INTO nr_tari
    FROM employees
    JOIN departments ON departments.department_id =
employees.department_id
    JOIN locations ON locations.location_id = departments.location_id
    WHERE employees.manager_id = id_ang;

    RETURN nr_tari;
END;
/

CREATE OR REPLACE FUNCTION F_SalMax(id_ang NUMBER)
RETURN NUMBER
AS
    sal_max NUMBER;
BEGIN
    SELECT MAX(max_salary)
    INTO sal_max
    FROM employees
    JOIN jobs ON employees.job_id = jobs.job_id
    WHERE manager_id = id_ang
        OR employee_id = id_ang;

    RETURN sal_max;
END;
/

CREATE OR REPLACE PROCEDURE NumeDeProcedura
AS
BEGIN
    dbms_output.put_line(RPAD('Nume Angajat', 50)
        || RPAD('Nr Subalterni', 15)
```

```
        || RPAD('Nr Funcii', 15)
        || RPAD('Diff Max Sal', 15)
        || RPAD('Nr Tari', 15)
        || LPAD('Max Sal', 15));
    dbms_output.put_line(RPAD('*', 50, '*')
        || RPAD('*', 15, '*')
        || RPAD('*', 15, '*')
        || RPAD('*', 15, '*')
        || RPAD('*', 15, '*')
        || LPAD('*', 15, '*'));
    FOR emp IN (
        SELECT first_name || ' ' || last_name nume,
            F_Subalterni(employee_id) nr_sub,
            F_Functii(employee_id) nr_f,
            F_DiffMax(department_id) diff_m,
            F_TariDistincte(employee_id) tari_d,
            F_SalMax(employee_id) sal_m
        FROM employees
    )
    LOOP
        dbms_output.put_line(RPAD(emp.nume, 50)
            || RPAD(emp.nr_sub, 15)
            || RPAD(emp.nr_f, 15)
            || RPAD(emp.diff_m, 15)
            || RPAD(emp.tari_d, 15)
            || LPAD(emp.sal_m, 15));
    END LOOP;
END;
/
BEGIN
    NumeDeProcedura();
END;
```

ExLab05Bonus

```
/*
Sa se afiseze folosind minim o functie si minim o procedura stocata
urmatoarea lista:
- nume angajat
- daca este subalternul unui manager de departament ('DA/NU')
- daca este subalternul managerului cu numarul maxim de subaterni
pentru departamentul lui ('DA/NU')
- cate salarii sunt mai mari decat al lui pentru functia pe care o
detine
- cate salarii sunt mai mici decat al lui pentru functia pe care o
detine
- daca este peste media salariilor din tara lui ('DA/NU')
*/
```

```
WITH emp_per_manager
     AS (SELECT manager_id,
                department_id,
                Count(*) emp_count
         FROM   employees
         GROUP  BY manager_id,
                   department_id),
     emp_max_per_dept
     AS (SELECT department_id,
                Max(emp_count) max_count
         FROM   emp_per_manager
         GROUP  BY department_id),
     man_with_max_emp
     AS (SELECT manager_id
         FROM   emp_per_manager
         WHERE  ( emp_count, department_id ) IN (SELECT max_count,
                                                        department_id
                                                 FROM
emp_max_per_dept))
SELECT e.first_name
       || ' '
       || e.last_name                NAME,
       CASE
         WHEN e.manager_id IN (SELECT DISTINCT manager_id
                               FROM   departments
                               WHERE  manager_id IS NOT NULL) THEN
'DA'
         ELSE 'NU'
       END                           IsUnderDeptManager,
       CASE
         WHEN e.manager_id IN (SELECT manager_id
                               FROM   man_with_max_emp) THEN 'DA'
         ELSE 'NU'
       END                           IsUnderCoolManager,
       (SELECT Count(*)
        FROM   employees
        WHERE  salary < e.salary
               AND job_id = e.job_id) LessThan,
       (SELECT Count(*)
        FROM   employees
        WHERE  salary > e.salary
               AND job_id = e.job_id) GreaterThan,
       CASE
         WHEN e.salary > (SELECT Avg(salary)
                          FROM   employees esq
                                 join departments dsq
                                   ON esq.department_id =
dsq.department_id
                                 join locations lsq
                                   ON lsq.location_id =
dsq.location_id
```

```
                                        AND l.country_id =
lsq.country_id) THEN
         'DA'
         ELSE 'NU'
       END                              IsAboveForCountry
FROM   employees e
join departments d ON e.department_id = d.department_id
join locations l ON l.location_id = d.location_id;
```

EXRezolvat

```
/* SÄƒ se obÈ›inÄƒ o listÄƒ cu urmÄƒtorul conÈ›inut:
1P   - nume È™ef = concatenarea prenumelui cu numele (un È™ef este o
persoanÄƒ al cÄƒrui employee_id se gÄƒseÈ™te pe coloana manager_id)
0.5P - venit È™ef (venit = salariu * (1 + puncte commision))
1P   - DacÄƒ este sau nu manager de departament = 'DA/NU' (dacÄƒ id-ul
lui apare Ã®n coloana manager_id din departments)
5P   - numÄƒrul de subalterni, ierarhic (DacÄƒ A este manager pentru B
È™i C, B are x subalterni È™i C are Y atunci A are x + y + 2)
2.5P - bonus = venit * procent de bonus:
dacÄƒ nr_subalterni >= 10 È™i este manager de departament 15%
dacÄƒ nr_subalterni este Ã®ntre 6 È™i 10 È™i este manager de
departament atunci procentul de bonus este 20%
dacÄƒ nr_subalterni < 6 È™i este manager de departament atunci
procentul de bonus este 10%
dacÄƒ nr_subalterni < 6 È™i nu este manager de departament procentul
de bonus este 15%
altfel procentul de bonus este 5%
*/

SET SERVEROUTPUT ON;
DECLARE
    TYPE INFO_SEF IS RECORD(
        M_ID EMPLOYEES.EMPLOYEE_ID%TYPE,
        NUME VARCHAR(46),
        VENIT_SEF EMPLOYEES.SALARY%TYPE,
        NR_SUBALTERNI NUMBER
    );
    TYPE LISTA_INFO_SEF IS TABLE OF INFO_SEF;

    LISTA LISTA_INFO_SEF;
    contor INTEGER;

    E_MGR VARCHAR(2);
    BONUS NUMBER(8, 2);
BEGIN
```

```
    SELECT M.EMPLOYEE_ID,
        M.FIRST_NAME || ' ' || M.LAST_NAME,
        M.SALARY * (1 + NVL(m.commission_pct, 0)),
        COUNT(M.EMPLOYEE_ID)
    BULK COLLECT INTO LISTA
    FROM EMPLOYEES E
    JOIN EMPLOYEES M ON E.MANAGER_ID = m.employee_id
    GROUP BY M.EMPLOYEE_ID, M.FIRST_NAME || ' ' || M.LAST_NAME,
        M.SALARY * (1 + NVL(m.commission_pct, 0))
    ORDER BY 1;

    dbms_output.PUT_LINE(RPAD('NUME', 46) || RPAD('VENIT', 15) ||
RPAD('E_MGR', 10) || RPAD('SUBALTERNI', 15) || RPAD('BONUS', 15));
    dbms_output.PUT_LINE(RPAD('=', 46, '=') || RPAD('=', 15, '=') ||
RPAD('=', 10, '=') || RPAD('=', 15, '=') || RPAD('=', 15, '='));
    FOR contor IN LISTA.FIRST..LISTA.LAST
    LOOP
        E_MGR := 'DA';
        BEGIN
            SELECT 1
            INTO BONUS
            FROM DEPARTMENTS
            WHERE MANAGER_ID = LISTA(contor).M_ID;
        EXCEPTION
            WHEN OTHERS THEN
                E_MGR := 'NU';
        END;

        BONUS := CASE
            WHEN ((LISTA(contor).NR_SUBALTERNI >= 10) AND (E_MGR =
'DA')) THEN LISTA(contor).VENIT_SEF * 0.15
            ELSE LISTA(contor).VENIT_SEF * 0.05
        END;

        dbms_output.PUT_LINE(RPAD(LISTA(contor).NUME, 46) ||
RPAD(LISTA(contor).VENIT_SEF, 15) || RPAD(E_MGR, 10) ||
RPAD(LISTA(contor).NR_SUBALTERNI, 15) || LPAD(BONUS, 15));
    END LOOP;
END;
```

RecapOracle

```
/* Toti angajatii */
SELECT employee_id,
       first_name,
       last_name,
       email,
       phone_number,
       hire_date,
       job_id,
       salary,
       commission_pct,
       manager_id,
       department_id
FROM   employees;

/* Select standard
[WITH] --CTE
SELECT
[FROM] -- Sursa sau sursele de date de date
[[[CROSS]|[INNER]|[LEFT|RIGHT|FULL OUTER]] JOIN] -- Putem folosi mai
multe surse de date cu condi?ii speciale
[WHERE] -- Condi?iile de filtrare a datelor deja existente
[GROUP BY] -- Agregarea datelor deja existente pânã în acest punct
[HAVING] -- Condi?ii pentru noile grupãri
[ORDER BY] -- Ordonare, se pot folosi alias  * uri
*/
/* Incercarea 1 : angajati cu mangerii lor */
SELECT e.*,
       m.*
FROM   employees e,
       employees m
WHERE  e.manager_id = m.employee_id;

/* Incercaraea 2 : inner join */
SELECT     e.*,
           m.*
FROM       employees e
inner join employees m
ON         e.manager_id = m.employee_id;

/* Incercarea 3 : angajati cu mangerii lor */
SELECT    e.*,
          m.*
FROM      employees e
left join employees m
ON        e.manager_id = m.employee_id;

/* Incercarea 4 : angajati cu mangerii lor, folosind RIGHT */
SELECT     e.*,
```

```
          m.*
FROM      employees e
right join employees m
ON        e.employee_id = m.manager_id;

/* Subclauza: Toti angajatii suboptim level 1*/
SELECT first_name
             || ' '
             || last_name
FROM   employees
WHERE  1 IN
       (
              SELECT 1
              FROM   employees)
fetch NEXT 2 ROWS ONLY;

/* Subclauze: Vrem nume si prenume pentru toti angajatii care sunt
manageri */
--multumim Sergiu  + (Alin, mentiune)
SELECT first_name
             ||' '
             ||last_name "Name"
FROM   employees e
WHERE  e.employee_id IN
                        (
                        SELECT DISTINCT m.manager_id
                        FROM            employees m
                        WHERE           m.manager_id IS NOT NULL);

/* JOIN: Vrem nume si prenume pentru toti angajatii care sunt manageri
*/
SELECT DISTINCT m.first_name
                                || '  '
                                || m.last_name "Name"
FROM            employees e
left join       employees m
ON              e.manager_id = m.employee_id
WHERE           e.manager_id IS NOT NULL;

/* Vrem nume si preume pentru toti angajatii care au salariul maxim
din departamentul lor */
-- 1. Obtinem salariul maxim pe fiecare departament
select max(salary), department_id
from employees
group by department_id;

-- 2. Unificam clauza anterioara cu selectia din employees a ce ne
intereseaza
select first_name || ' ' || last_name NAME
from employees
where (department_id, salary) IN (select department_id, max(salary)
                                  from employees
```

```
                                    group by department_id);

/* Vrem angajatii din departamentul cu numarul maxim de angajati */
-- 1. Numarul de angajati din fiecare departament
select count(*) emp_count, department_id
from employees
group by department_id;

-- 2. Numarul maxim de angajati
select max(emp_count)
from (
    select count(*) emp_count, department_id
    from employees
    group by department_id);

-- 3. Departamentul cu numarul maxim de angajati
SELECT Count(*) emp_count,
       department_id
FROM   employees
GROUP  BY department_id
HAVING Count(*) = (SELECT Max(emp_count)
                   FROM   (SELECT Count(*) emp_count,
                                  department_id
                           FROM   employees
                           GROUP  BY department_id));

-- 3.a CTE
WITH emp_count_per_d
     AS (SELECT Count(*) emp_count,
                department_id
         FROM   employees
         GROUP  BY department_id)
SELECT emp_count,
       department_id
FROM   emp_count_per_d
WHERE  emp_count = (SELECT Max(emp_count)
                    FROM   emp_count_per_d);

-- 4 Angajatii din acel departament
WITH emp_count_per_d
     AS (SELECT Count(*) emp_count,
                department_id
         FROM   employees
         GROUP  BY department_id)
SELECT *
FROM   employees
WHERE  department_id IN (SELECT department_id
                         FROM   emp_count_per_d
                         WHERE  emp_count = (SELECT Max(emp_count)
                                             FROM   emp_count_per_d));

/* Vrem angajatii care au functia cu numarul maxim de angajati */
```

```
WITH emp_count_per_j
     AS (SELECT Count(*) emp_count,
                JOB_ID
         FROM   employees
         GROUP  BY JOB_ID)
SELECT *
FROM   employees
WHERE  JOB_ID IN (SELECT JOB_ID
                         FROM   emp_count_per_j
                         WHERE  emp_count = (SELECT Max(emp_count)
                                             FROM   emp_count_per_j));

/* Vrem angajatii care detin o functie pentru care salariul mediu pe
acea
functie este diferit cu maxim 20% (modul) fata de salariul mediu din
companie */
WITH avg_sal_per_j
     AS (SELECT Avg(salary) avg_sal,
                job_id
         FROM   employees
         GROUP  BY job_id),
     avg_sal
     AS (SELECT Avg(salary) avg_sal
         FROM   employees)
SELECT *
FROM   employees
WHERE  job_id IN (SELECT job_id
                  FROM   avg_sal_per_j aj,
                         avg_sal a
                  WHERE  ( ( Abs(aj.avg_sal - a.avg_sal) / a.avg_sal )
< 0.2 ));

/*
Sa se afiseze urmatoarea lista:
- nume angajat
- daca este subalternul unui manager de departament ('DA/NU')
- daca este subalternul managerului cu numarul maxim de subaterni
pentru departamentul lui ('DA/NU')
- cate salarii sunt mai mari decat al lui pentru functia pe care o
detine
- cate salarii sunt mai mici decat al lui pentru functia pe care o
detine
- daca este peste media salariilor din tara lui ('DA/NU')
*/
WITH emp_per_manager
     AS (SELECT manager_id,
                department_id,
                Count(*) emp_count
         FROM   employees
         GROUP  BY manager_id,
                   department_id),
     emp_max_per_dept
```

```
     AS (SELECT department_id,
                Max(emp_count) max_count
         FROM   emp_per_manager
         GROUP  BY department_id),
     man_with_max_emp
     AS (SELECT manager_id
         FROM   emp_per_manager
         WHERE  ( emp_count, department_id ) IN (SELECT max_count,
                                                        department_id
                                                 FROM
emp_max_per_dept))
SELECT e.first_name
       || ' '
       || e.last_name                  NAME,
       CASE
         WHEN e.manager_id IN (SELECT DISTINCT manager_id
                               FROM   departments
                               WHERE  manager_id IS NOT NULL) THEN
'DA'
         ELSE 'NU'
       END                             IsUnderDeptManager,
       CASE
         WHEN e.manager_id IN (SELECT manager_id
                               FROM   man_with_max_emp) THEN 'DA'
         ELSE 'NU'
       END                             IsUnderCoolManager,
       (SELECT Count(*)
        FROM   employees
        WHERE  salary < e.salary
               AND job_id = e.job_id) LessThan,
       (SELECT Count(*)
        FROM   employees
        WHERE  salary > e.salary
               AND job_id = e.job_id) GreaterThan,
       CASE
         WHEN e.salary > (SELECT Avg(salary)
                          FROM   employees esq
                                 join departments dsq
                                   ON esq.department_id =
dsq.department_id
                                 join locations lsq
                                   ON lsq.location_id =
dsq.location_id
                                      AND l.country_id =
lsq.country_id) THEN
         'DA'
         ELSE 'NU'
       END                             IsAboveForCountry
FROM   employees e
join departments d ON e.department_id = d.department_id
join locations l ON l.location_id = d.location_id;
/
```

```
/*
Sa se afiseze folosind minim o functie si minim o procedura stocata
urmatoarea lista:
- nume angajat 0.5
- daca este subalternul unui manager de departament ('DA/NU') 1.5
- daca este subalternul managerului cu numarul maxim de subaterni
pentru departamentul lui ('DA/NU') 2.5
- cate salarii sunt mai mari decat al lui pentru functia pe care o
detine 1.5
- cate salarii sunt mai mici decat al lui pentru functia pe care o
detine 1.5
- daca este peste media salariilor din tara lui ('DA/NU') 1.5
*/
CREATE OR REPLACE FUNCTION F_Than(p_salary NUMBER, p_job_id VARCHAR2,
is_greater INTEGER)
RETURN INTEGER
AS
    emp_count INTEGER;
BEGIN
    SELECT Count(*)
    INTO emp_count
    FROM   employees
    WHERE  (is_greater = 1 AND (salary < p_salary ))
        OR (is_greater = 0 AND (salary > p_salary ))
    AND job_id = p_job_id;

    RETURN emp_count;
END;
/
CREATE OR REPLACE PROCEDURE Afisare1
AS
BEGIN
FOR ang in (WITH emp_per_manager
         AS (SELECT manager_id,
                    department_id,
                    Count(*) emp_count
             FROM   employees
             GROUP  BY manager_id,
                       department_id),
         emp_max_per_dept
         AS (SELECT department_id,
                    Max(emp_count) max_count
             FROM   emp_per_manager
             GROUP  BY department_id),
         man_with_max_emp
         AS (SELECT manager_id
             FROM   emp_per_manager
             WHERE  ( emp_count, department_id ) IN (SELECT max_count,

department_id
                                                     FROM
emp_max_per_dept))
```

```
    SELECT e.first_name
           || ' '
           || e.last_name                 NAME,
           CASE
             WHEN e.manager_id IN (SELECT DISTINCT manager_id
                                   FROM   departments
                                   WHERE  manager_id IS NOT NULL) THEN
'DA'
             ELSE 'NU'
           END                            IsUnderDeptManager,
           CASE
             WHEN e.manager_id IN (SELECT manager_id
                                   FROM   man_with_max_emp) THEN 'DA'
             ELSE 'NU'
           END                            IsUnderCoolManager,
           F_Than(e.salary, e.job_id, 1) LessThan,
           F_Than(e.salary, e.job_id, 0) GreaterThan,
           CASE
             WHEN e.salary > (SELECT Avg(salary)
                              FROM   employees esq
                                     join departments dsq
                                       ON esq.department_id =
dsq.department_id
                                     join locations lsq
                                       ON lsq.location_id =
dsq.location_id
                                          AND l.country_id =
lsq.country_id) THEN
             'DA'
             ELSE 'NU'
           END                            IsAboveForCountry
    FROM   employees e
    join departments d ON e.department_id = d.department_id
    join locations l ON l.location_id = d.location_id)
LOOP
    dbms_output.put_line(RPAD(ang.NAME, 45)
    || RPAD(ang.IsUnderDeptManager, 20)
    || RPAD(ang.IsUnderCoolManager, 20)
    || RPAD(ang.LessThan, 20)
    || RPAD(ang.GreaterThan, 20)
    || RPAD(ang.IsAboveForCountry, 20));
END LOOP;
END;
/

SET SERVEROUTPUT ON;

BEGIN
    Afisare1();
END ;
```

```
/* Sa se scrie un TRIGGER care monitorizeaza schimbarile pe tabela
employees pentru a adauga date in tabela job_history astfel:
- dorim sa stim cand un angajat isi schimba job-ul (1P) (+1P
instructiunea care demonstreaza functionalitatea)
- dorim sa stim cand un angajat isi schimba departament-ul (1P) (+1P
instructiunea care demonstreaza functionalitatea)
- dorim sa stim cand un angajat isi schimba si departament-ul si job-
ul (1P) (+1P instructiunea care demonstreaza functionalitatea)
- dorim sa stim cand un angajat isi da demisia (este sters) (1P) (+1P
instructiunea care demonstreaza functionalitatea)
- dorim sa stim cand apare un angajat nou (1P) (+1P instructiunea care
demonstreaza functionalitatea) */
CREATE OR REPLACE TRIGGER T_Job_Dept
AFTER INSERT OR DELETE OR UPDATE OF department_id, job_id
ON employees
FOR EACH ROW
DECLARE
BEGIN
    IF UPDATING('department_id') AND UPDATING('job_id') THEN
        INSERT INTO job_history (employee_id, start_date, end_date,
job_id, department_id)
        VALUES (:new.employee_id, sysdate, sysdate + 10, :new.job_id,
:new.department_id);
    ELSIF UPDATING('department_id') THEN
        INSERT INTO job_history (employee_id, start_date, end_date,
job_id, department_id)
        VALUES (:new.employee_id, sysdate, sysdate + 10, :new.job_id,
:new.department_id);
    ELSIF UPDATING('job_id') AND (:new.job_id <> :old.job_id) THEN
        INSERT INTO job_history (employee_id, start_date, end_date,
job_id, department_id)
        VALUES (:new.employee_id, sysdate, sysdate + 10, :new.job_id,
:new.department_id);
    ELSIF INSERTING THEN
        INSERT INTO job_history (employee_id, start_date, end_date,
job_id, department_id)
        VALUES (:new.employee_id, :new.hire_date, sysdate + 10,
:new.job_id, :new.department_id);
    ELSIF DELETING THEN
        INSERT INTO job_history (employee_id, start_date, end_date,
job_id, department_id)
        VALUES (:old.employee_id, :old.hire_date + 1, sysdate +10,
:old.job_id, :old.department_id);
    END IF;
END;
/

SELECT COUNT(*)
FROM job_history;

UPDATE employees
SET commission_pct = nvl(commission_pct, 0);
```

```
UPDATE employees
SET job_id = job_id;

insert into employees
values(707, 'Miss', 'Marple', 'crime@kill.org','07n-am cartela',
to_date('19-DEC-07'), 'AC_ACCOUNT', 8300, NULL, 205, 110);

-- ESTE PROBLEMATIC
DELETE FROM employees
WHERE employee_id = 707;
```

ReOracle

```
/* Sa se scrie un trigger care logheaza modificarile din jobs intr-o
tabela noua:
- insert
- update (a crecut max_salary)
- delete
*/
CREATE TABLE LOGS_OF_JOBS
(
 WHO varchar2(50),
 CHANGE_TYPE varchar2(50),
 CHANGE_DETAILS varchar2(1024),
 CHANGE_DATE DATE
);
/
CREATE OR REPLACE TRIGGER SALARY_WATCH_TRIGGER
AFTER INSERT OR DELETE OR UPDATE OF max_salary
ON jobs
FOR EACH ROW
DECLARE
    to_insert LOGS_OF_JOBS%ROWTYPE;
    is_valid_log INTEGER := 1;
BEGIN
    to_insert.WHO := user;
    to_insert.CHANGE_DATE := sysdate;

    IF INSERTING THEN
        to_insert.CHANGE_TYPE := 'Insert';
        to_insert.CHANGE_DETAILS := 'Adding a new row: ' ||
:new.job_title || ' with max_salary ' || :new.max_salary;
    ELSIF DELETING THEN
```

```
        to_insert.CHANGE_TYPE := 'Deleting';
        to_insert.CHANGE_DETAILS := 'Removing an existing row: ' ||
:old.job_title || ' with max_salary ' || :old.max_salary;
    ELSIF (:new.max_salary > :old.max_salary) THEN
        to_insert.CHANGE_TYPE := 'Updating';
        to_insert.CHANGE_DETAILS := 'Increase of max_salary: ' ||
:new.job_title || ' from max_salary of ' || :old.max_salary || ' to '
|| :new.max_salary;
    ELSE
        is_valid_log := 0;
    END IF;

    IF is_valid_log = 1 THEN
        INSERT INTO LOGS_OF_JOBS
        VALUES (to_insert.WHO, to_insert.CHANGE_TYPE,
to_insert.CHANGE_DETAILS, to_insert.CHANGE_DATE);
    END IF;
END;
/

SELECT *
FROM LOGS_OF_JOBS;

INSERT INTO JOBS
VALUES ('TST_VALUE', 'Just a Test', 0, 100);

SELECT *
FROM LOGS_OF_JOBS;

UPDATE JOBS
SET max_salary = max_salary + 100
WHERE job_id = 'TST_VALUE';

SELECT *
FROM LOGS_OF_JOBS;

DELETE FROM JOBS
WHERE job_id = 'TST_VALUE';

SELECT *
FROM LOGS_OF_JOBS;

UPDATE JOBS
SET max_salary = max_salary;

SELECT *
FROM LOGS_OF_JOBS;

/* Sa se scrie un pachet (0.5P) care contine minim o functie (0.5P) si
o procedura(0.5P) care genereaza o lista cu urmatoarea structura (si
sa se apeleze):
- numele job-ului (0.5P)
```

```
- cati angajati detin acest job (1P)
- cati manageri detin acest job (1.5P)
- cati sefi de departament detin acest job (1.5P)
- care este venitul mediu al persoanelor care detin acest job (2P)
- daca este atins salariul maxim pentru acest job = 'DA/NU'(2P) */
SELECT j.job_title,
    count(*) emp_count,
    sum(CASE WHEN e.employee_id IN (SELECT manager_id FROM employees)
THEN 1 ELSE 0 END) mgr_count,
    sum(CASE WHEN e.employee_id IN (SELECT manager_id FROM
departments) THEN 1 ELSE 0 END) dept_mgr_count,
    avg(e.salary * (1 + nvl(e.commission_pct, 0))) avg_wage,
    CASE WHEN j.max_salary = max(e.salary) THEN 'DA' ELSE 'NU' END
has_max
FROM jobs j
INNER JOIN employees e on j.job_id = e.job_id
GROUP BY j.job_title, max_salary;

CREATE OR REPLACE PACKAGE p_ex6 AS
        FUNCTION is_manager(emp_id INTEGER) RETURN INTEGER;
        FUNCTION is_dept_manager(emp_id INTEGER) RETURN INTEGER;

        PROCEDURE show_content;
END;
/

CREATE OR REPLACE PACKAGE BODY p_ex6 AS
        FUNCTION is_manager(emp_id INTEGER)
        RETURN INTEGER
        AS
            is_mgr INT;
        BEGIN
            SELECT COUNT(*)
            INTO is_mgr
            FROM EMPLOYEES
            WHERE manager_id = emp_id;

            IF is_mgr > 0 THEN
                RETURN 1;
            ELSE
                RETURN 0;
            END IF;
        END;

        FUNCTION is_dept_manager(emp_id INTEGER)
        RETURN INTEGER
        AS
            is_dept_mgr INT;
        BEGIN
            SELECT COUNT(*)
            INTO is_dept_mgr
            FROM DEPARTMENTS
```

```
            WHERE manager_id = emp_id;

            IF is_dept_mgr > 0 THEN
                RETURN 1;
            ELSE
                RETURN 0;
            END IF;
        END;

        PROCEDURE show_content
        AS
        BEGIN
            FOR job_info IN (SELECT j.job_title,
                                    count(*) emp_count,
                                    sum(p_ex6.is_manager(e.manager_id))
mgr_count,

sum(p_ex6.is_dept_manager(e.manager_id)) dept_mgr_count,
                                    ROUND(avg(e.salary * (1 +
nvl(e.commission_pct, 0))), 2) avg_wage,
                                    CASE WHEN j.max_salary = max(e.salary)
THEN 'DA' ELSE 'NU' END has_max
                                FROM jobs j
                                INNER JOIN employees e on j.job_id =
e.job_id
                                GROUP BY j.job_title, max_salary)
            LOOP
                dbms_output.put_line(RPAD(job_info.job_title, 30)
                || RPAD(job_info.mgr_count, 15)
                || RPAD(job_info.dept_mgr_count, 15)
                || RPAD(job_info.avg_wage, 15)
                || RPAD(job_info.has_max, 15));
            END LOOP;
        END;
END;
/

SET SERVEROUTPUT ON;

BEGIN
    p_ex6.show_content;
END;
```